Meta-otimização do ajuste heurístico em arquiteturas Minimax utilizando LLM Agent Loops.

Guilherme Borges Pagano
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
guilhermebpagano@ufu.br

João Marcos de Oliveira Calixto
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
jm.calixto@ufu.br

***Resumo* — A aplicação de inteligência artificial ao xadrez computacional tradicionalmente exige a calibração manual de funções de avaliação heurística. Esse processo é notoriamente trabalhoso, e difícil de escalar para modificações estruturais no código. Este artigo propõe um framework de meta-otimização que substitui o ajuste heurístico manual por uma orquestração automatizada. Inspirado no padrão Ralph Wiggum AI, um script de controle atua como um "usuário persistente", gerando tarefas e alimentando iterativamente um Large Language Model (LLM) com parâmetros de código e logs de desempenho. O modelo de linguagem, atuando estritamente como motor de mutação, propõe alterações no módulo de avaliação da engine. O orquestrador então compila o código e verifica a execução contra suítes de testes táticos estáticos (WAC, Kaufman), retendo apenas as alterações que produzem ganho mensurável de acurácia ou superioridade em partidas controladas contra versões anteriores (auto-jogo) e motores de referência. A arquitetura acopla um loop interno (busca adversarial Minimax) a um loop externo (otimização orientada por feedback verificável), tratando o ajuste heurístico como um problema de busca em espaço de programas. Utilizando a engine ChessLib desenvolvida em Rust como baseline, o sistema demonstra que a delegação do prompt iterativo a um controlador automatizado permite ao LLM melhorar progressivamente a qualidade da decisão tática sem retreino dos pesos do modelo. Os resultados experimentais comparam a versão otimizada contra a baseline em métricas de acerto posicional e ganho de rating relativo, discutindo riscos de overfitting às suítes de teste e estratégias de mitigação.**

***Palavras chave*: Inteligência Artificial, Xadrez, Meta-otimização, Agent Loop;**

I. Introdução

O xadrez constitui um problema canônico para a inteligência artificial e a teoria da decisão adversarial. Formalmente classificado como um jogo determinístico, de informação perfeita, finito e de soma zero para dois jogadores [1], seu espaço de estados é computacionalmente colossal: a árvore de jogo possui um limite inferior estimado em 10^{120} permutações únicas, o chamado Número de Shannon [2]. Essa explosão combinatória inviabiliza a busca exaustiva desde a posição inicial até os estados terminais, tornando obrigatório o uso de mecanismos de aproximação e poda.

A arquitetura clássica para superar essa barreira combina o algoritmo Minimax com a poda alfa-beta [3], [4]. O Minimax explora a árvore de jogo assumindo que ambos os jogadores atuam de forma ótima, enquanto a poda alfa-beta elimina ramos que não podem influenciar a decisão final, reduzindo drasticamente o número de nós avaliados, conforme analisado formalmente por [5]. Contudo, como a profundidade da busca é limitada por horizonte computacional, a avaliação de posições não-terminais depende de uma função heurística de avaliação estática – tipicamente uma combinação linear de material, tabelas posicionais (piece-square tables) [6], mobilidade e outros termos de conhecimento de xadrez.

A calibração dessa função heurística é um dos pontos mais trabalhosos e frágeis do desenvolvimento de engines de xadrez. O ajuste manual de pesos, matrizes e até mesmo a introdução ou remoção de features exige extensivas rodadas de partidas contra versões anteriores ou engines de referência. O problema é agravado pelo fato de que o "sinal" de qualidade é observado por resultados de partidas – uma medida ruidosa, sujeita à variância de abertura, controle de tempo e sorte residual. Métodos estatísticos (como otimização estocástica e abordagens bayesianas) automatizam parte desse processo, mas ainda operam dentro de um espaço paramétrico pré-definido, raramente alterando a estrutura do código heurístico.

Nos últimos anos, a engenharia de software passou a documentar evidências de que grandes modelos de linguagem (LLMs) conseguem melhorar código iterativamente quando existe um juiz verificável – tipicamente um compilador, uma suíte de testes ou um harness de validação executável. O benchmark SWE-bench formaliza essa avaliação por patches verificados por testes em repositórios reais [7], e sistemas agentes como SWE-agent mostram que feedback ambiente e validação por testes aumentam a taxa de resolução sem necessidade de retreinar os pesos do modelo [8]. Paralelamente, frameworks que acoplam LLMs a avaliadores sistemáticos têm evoluído programas em direção a melhores scores em domínios matemáticos e de otimização – como FunSearch na descoberta de soluções em matemática discreta [9] e SBLLM na otimização de código guiada por busca e refinamento iterativo [10].

A hipótese central deste artigo é que essa lógica de gerar → verificar → corrigir pode ser transplantada para o ajuste tático de heurísticas em engines clássicas de xadrez. Em vez de ajustar pesos "à mão" ou exclusivamente por otimização estocástica, é proposto um framework de meta-otimização onde um script orquestrador persistente submete o módulo de avaliação a um LLM e valida as mutações através de feedback verificável do compilador, de testes de regressão e de um avaliador de desempenho (ex.: acurácia em posições táticas ou resultado contra um motor de referência). O ajuste heurístico passa a ser tratado

como um problema de busca em espaço de programas, análogo às abordagens recentes que combinam LLMs com avaliadores sistemáticos.

II. Trabalhos Relacionados

Esta seção contextualiza a proposta frente a três linhas de pesquisa:

1) O ajuste automático de heurísticas e parâmetros em motores de xadrez clássicos;
2) O uso de grandes modelos de linguagem (LLMs) para geração, correção e otimização autônoma de código;
3) Arquiteturas de agentes com loops de feedback verificável e iteração contínua.

A síntese dessas vertentes delimita a lacuna explorada pelo artigo: a meta-otimização de funções de avaliação heurística em motores Minimax por meio de um agente LLM em loop, com validação por compilação, testes e desempenho tático.

A. Ajuste automático de heurísticas e parâmetros em motores clássicos

A literatura de tuning de motores de xadrez trata explicitamente o problema como otimização sob ruído, pois a função objetivo (força de jogo) é observada por resultados de partidas – medida com variância alta, dependente de aberturas, controle de tempo e sorte residual. Nesse cenário, o método SPSA (Simultaneous Perturbation Stochastic Approximation) é frequentemente utilizado por sua eficiência em espaços de alta dimensionalidade, estimando o gradiente com poucas avaliações por iteração [11]. Há também propostas recentes que exploram inferência bayesiana para melhorar a eficiência e estabilidade do processo, mantendo a avaliação por confronto entre variantes [12].

Além disso, práticas difundidas na comunidade – embora nem sempre formalizadas em papers – incluem métodos de regressão logística para calibrar pesos de avaliação a partir de posições rotuladas por resultado ou score, conhecido como Texel Tuning [13]. Esses métodos reforçam que o ajuste "automático" já é parte do estado da prática, mas tipicamente restrito a vetores de parâmetros em um modelo heurístico pré-definido, em vez de alterações estruturais no código ou introdução de novas features.

B. LLMs para geração, correção e otimização autônoma de código

Em engenharia de software, a virada metodológica recente está na avaliação por artefatos verificáveis. O benchmark SWE-bench define tarefas reais extraídas de issues e pull requests em repositórios, julgando a correção pela execução de testes [7]. Esse desenho aproxima a avaliação do que um desenvolvedor faria: modificar código e validar por suíte de testes. Sobre esse benchmark, sistemas agentes interativos como SWE-agent argumentam que o desempenho depende criticamente do "acoplamento" agente–ambiente: ações de navegação/edição e feedback conciso por turno melhoram a capacidade de iterar até passar nos testes [8].

Em paralelo, a subárea de reparo automatizado de programas (Automated Program Repair, APR) documenta um padrão convergente: LLMs podem ser integrados a ciclos generate-and-validate, mas ganhos importantes aparecem quando o loop incorpora feedback de falhas de teste (ou de compilação) para guiar novas tentativas, como em abordagens conversacionais que intercalam geração e feedback ao invés de apenas amostrar múltiplos patches independentes [14]. Linhas próximas também exploram feedback de compilador para reduzir erros de contexto e de uso de APIs em geração de código em nível de projeto [15].

C. Arquiteturas de agentes com loops de feedback verificável e iteração contínua

O deslocamento de "uma chamada do LLM" para "um agente em loop" é sustentado por frameworks de raciocínio e ação e de auto-refinamento. ReAct formaliza a alternância entre raciocínio e ações no ambiente [16]; Self-Refine modela iteração crítica/refinamento [17]; Reflexion propõe incorporar feedback em "memória" textual para melhorar tentativas subsequentes sem ajuste de pesos [18]. Frameworks de deliberação como Tree of Thoughts generalizam a ideia para exploração de múltiplos caminhos, aproximando o processo de uma busca em espaço de soluções [19].

Em termos de infraestrutura, plataformas como OpenHands sistematizam a interação de agentes com repositórios, terminal e benchmarks, reforçando a noção de "ambiente verificável" [20]. Um padrão de engenharia particularmente relevante para este artigo é o Ralph Wiggum AI – um loop externo persistente no qual cada iteração executa uma unidade de trabalho, e um verificador decide se a tarefa terminou; caso contrário, injeta feedback e o agente tenta novamente [21], [22].

D. Síntese e a lacuna explorada

A lacuna explorada por este artigo não é "usar LLMs para jogar xadrez", mas usar LLMs para otimizar o código de um motor clássico – em especial sua função de avaliação heurística. Isso conecta diretamente (i) tuning sob ruído (partidas), (ii) geração/reparo de código com validação por testes/compilação, e (iii) loops agentes com avaliação externa. Trabalhos como FunSearch mostram que emparelhar LLM com avaliadores sistemáticos permite evoluir programas rumo a melhores scores [9]; SBLLM mostra que otimização de código pode ser formulada "como busca" com refinamento iterativo guiado por execução [10]; e abordagens recentes de cooperação compilador–LLM explicitam validação de correção como parte do loop [15].

Diferentemente desses trabalhos, o foco atual recai sobre um domínio adversarial de busca (Minimax + poda alfa-beta), onde o feedback de desempenho é inerentemente ruidoso e o espaço de modificações inclui tanto parâmetros contínuos quanto alterações estruturais no código da heurística.

III. Arquitetura

Este artigo propõe um sistema de meta-otimização organizado em dois níveis acoplados: um loop interno de busca adversarial (Minimax com poda alfa-beta) e um loop externo persistente no qual um agente baseado em LLM modifica o código da função de avaliação heurística, verifica a correção (compilação

+ testes) e mensura o ganho de desempenho (acurácia tática ou força relativa). A seguir, é detalhado cada componente e as decisões de projeto que viabilizam o ajuste heurístico como um problema de busca em espaço de programas.

A. Acoplamento de Loops

A arquitetura central pode ser apresentada como um acoplamento entre:

a) *Loop interno (busca adversarial)::*

Implementado pela engine ChessLib [23], o algoritmo Minimax com poda alfa-beta explora a árvore de jogo até uma profundidade limite, utilizando uma função de avaliação estática $E(s)$ para posições não-terminais. Essa função combina material, tabelas posicionais (piece-square tables) e, opcionalmente, outros termos heurísticos. O loop interno opera sobre um estado do tabuleiro e retorna o melhor lance encontrado.

b) *Loop externo (meta-otimização)::*

Opera sobre o artefato "código/heurísticas" (parâmetros, matrizes, features) e busca melhorias mensuráveis em um critério objetivo. O gerador é um LLM que propõe mutações no código; o avaliador é composto por (i) compilador (verificação sintática), (ii) testes de regressão (validação funcional) e (iii) benchmark tático ou partidas (medição de desempenho). O loop externo trata o ajuste heurístico como um problema de busca em espaço de programas, analogamente a frameworks que combinam LLMs com avaliadores sistemáticos [9], [10].

Essa separação é coerente com a visão clássica de "busca + avaliação" em jogos determinísticos: o loop interno estima o valor posicional sob um horizonte, enquanto o loop externo ajusta como essa estimativa é produzida, mantendo a engine como um sistema determinístico verificável por compilação e testes.

B. Loop externo com Ralph Wiggum AI

Para implementar o loop externo de forma robusta e autônoma, adotamos o padrão Ralph Wiggum AI – um loop persistente no qual cada iteração executa uma unidade de trabalho e um verificador decide se a tarefa foi concluída; caso contrário, injeta feedback e o agente tenta novamente [21], [22]. No contexto deste artigo, a sequência de tarefas de otimização é gerada e executada sob essa técnica, cujo nome é inspirado no personagem de The Simpsons e popularizado como um padrão de "autonomia contínua".

Operacionalmente, o loop externo segue estas etapas:

a) *Plano:*

O LLM (ou um orquestrador) produz uma micro-tarefa – por exemplo, "ajustar pesos da tabela posicional do cavalo no meio-jogo", "refatorar termo de segurança do rei" ou "inserir feature de mobilidade com custo $O(1)$".

b) *Execução:*

A tarefa é aplicada ao código (edição de arquivos fonte).

c) *Verificação:*

- Compilação (garantir que o código permanece sintaticamente correto).
- Execução de testes de regressão (evitar quebras funcionais).
- Avaliação de desempenho (acurácia em suíte tática ou partidas contra baseline).

d) *Decisão:*

Se todos os critérios forem satisfeitos, a mudança é "promovida" (commit/merge). Caso contrário, o feedback (erros de compilação, falhas em testes, ou ausência de melhoria) é realimentado ao LLM para uma nova tentativa.

C. Interface agente–repositório e feedback verificável

A literatura recente sobre agentes de software sugere que confiabilidade e desempenho dependem de restringir e instrumentar a interação com o ambiente. O SWE-agent introduz o conceito de agent-computer interface (ACI) para oferecer um conjunto pequeno de ações (ver arquivo, buscar, editar trechos) e retornar feedback conciso e estruturado a cada turno [8]. Essa ideia é diretamente transferível ao nosso contexto: em vez de permitir que o LLM edite qualquer coisa sem controle, a meta-otimização limita as mudanças ao módulo de avaliação/heurísticas, exige formatação específica (ex.: diffs ou blocos de código) e compila a cada commit.

Do lado do compilador, trabalhos que usam feedback de compilação/execução para orientar otimização mostram que "compilável" e "correto" podem ser tratados como restrições duras no loop, e que a retroalimentação do toolchain pode ser incorporada como sinal para nova tentativa, em vez de apenas amostragem independente [15].

Na atual implementação, o agente interage com um repositório Git local que contém o código fonte da engine ChessLib. Cada tentativa de modificação gera um branch temporário, dispara o pipeline de compilação e testes, e registra as métricas de desempenho. O feedback textual (erros do compilador, assertions falhas, acurácia tática) é enviado ao LLM para subsidiar a próxima iteração.

D. Critério de aceitação: melhoria relativa contra baseline e motor de referência

A forma mais defensável de definir "melhoria" no loop externo é por comparação pareada com a versão anterior (baseline) sob condições idênticas. Isso espelha a metodologia de testes A/B amplamente utilizada no desenvolvimento de motores de xadrez. O ecossistema do Stockfish e seu sistema de testes Fishtest explicitam o uso de testes sequenciais (SPRT/GSPRT) para decidir se uma mudança é melhor que outra com economia de recursos [24]. No presente artigo, define-se dois critérios complementares:

1) Acurácia tática: ganho positivo na suíte de posições (WAC, Kaufman) com significância estatística (teste de McNemar).
2) Força em partidas: vitória contra a versão baseline em um número controlado de partidas (ou evidência favorável em teste sequencial).

Formalmente, seja E_{base} a versão original e E_{novo} a versão após uma modificação, define-se:

- $\Delta\text{Acc}(T) = \text{Acc}(E_{\text{novo}}, T) - \text{Acc}(E_{\text{base}}, T)$, onde $\text{Acc}(E, T)$ é a acurácia em uma suíte tática T com tempo fixo por posição;
- $\Delta\text{Elo} = \text{elo}(E_{\text{novo}}) - \text{elo}(E_{\text{base}})$, estimado por partidas pareadas.

A mudança é aceita se $\Delta\text{Acc} > 0$ com $p < 0.05$ (McNemar) ou se o teste sequencial cruzar o limite de aceitação para $\Delta\text{Elo} > 0$ [25]. Rejeita-se caso contrário, e o feedback negativo é utilizado para guiar a próxima tentativa do LLM.

Essa estratégia de aceitação/rejeição, combinada com a persistência do loop Ralph Wiggum, permite que o agente evolua a heurística de forma incremental, evitando regressões e acumulando ganhos verificáveis.

IV. Implementação

V. Avaliação Experimental

VI. Resultados

VII. Discussão

VIII. Conclusão

Referências

[1] J. von Neumann e O. Morgenstern, *Theory of Games and Economic Behavior*. Princeton University Press, 1944.

[2] C. E. Shannon, “Programming a Computer for Playing Chess”, 1950.

[3] Chess Programming Wiki, “Minimax”. [Online]. Disponível em: https://www.chessprogramming.org/Minimax

[4] Chess Programming Wiki, “Alpha-Beta”. [Online]. Disponível em: https://www.chessprogramming.org/Alpha-Beta

[5] D. E. Knuth e R. W. Moore, “An Analysis of Alpha-Beta Pruning”, 1975.

[6] Chess Programming Wiki, “Piece-Square Tables”. [Online]. Disponível em: https://www.chessprogramming.org/Piece-Square_Tables

[7] C. E. Jimenez e others, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, 2024.

[8] J. Yang e others, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”, 2024.

[9] B. Romera-Paredes e others, “Mathematical discoveries from program search with large language models”, 2023.

[10] S. Gao e others, “SBLLM: Search-Based LLM for Code Optimization”, 2024.

[11] J. C. Spall, “Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation”, 1992.

[12] I. Ivec e I. Vojnović, “Bayesian Optimization of Chess Engine Parameters”, 2022.

[13] Chess Programming Wiki, “Texel's Tuning Method”. [Online]. Disponível em: https://www.chessprogramming.org/Texel%27s_Tuning_Method

[14] C. S. Xia e L. Zhang, “Conversational Automated Program Repair”, 2023.

[15] D. Grubisic e others, “Compiler-Guided LLM Optimization Loops”, 2024.

[16] S. Yao e others, “ReAct: Synergizing Reasoning and Acting in Language Models”, 2022.

[17] A. Madaan e others, “Self-Refine: Iterative Refinement with Self-Feedback”, 2023.

[18] N. Shinn e others, “Reflexion: Language Agents with Verbal Reinforcement Learning”, 2023.

[19] S. Yao e others, “Tree of Thoughts: Deliberate Problem Solving with Large Language Models”, 2023.

[20] OpenHands Team, “OpenHands: An Open Platform for AI Software Developers”, 2024.

[21] Vercel Labs, “Ralph Loop Agent”. [Online]. Disponível em: https://github.com/vercel-labs/ralph-loop-agent

[22] G. Huntley, “Ralph Wiggum AI – The Bash Loop Pattern”. [Online]. Disponível em: https://ralph-wiggum.ai/

[23] G. B. Pagano e J. M. Calixto, “Desenvolvimento de um Gerador de Lances para Xadrez em Rust com Bitboards e Magic Bitboards”, 2026.

[24] Stockfish Team, “Fishtest Mathematics”. [Online]. Disponível em: https://official-stockfish.github.io/docs/fishtest-wiki/Fishtest-Mathematics.html

[25] Chess Programming Wiki, “Sequential Probability Ratio Test”. [Online]. Disponível em: https://www.chessprogramming.org/SPRT